

# Working with Data

**Session 3 — Can I trust the answer?**

**Block 5 — Pipelines: bronze, silver, gold — cleaning as code**

## Knowledge check 3 — 10 minutes, closed book, 3 questions

- Covers **Session 2 and Homework 2**: joins, grain, the counts, JSON into tables
- One scenario, three questions. Each can be answered on its own
- One question is a query you write by hand. It is scored clause by clause
- Closed book means closed: no laptop, no phone, no notes
- **This block's lecture is 35 minutes**, not 45: the check comes out of the lecture, never out of the lab

## This block in one sentence

**Raw data is never edited. Every layer is made by code from the layer below, and every run makes the same tables.**

Bronze is what arrived. Silver is what you can trust. Gold is the answer.

And today's version of the course's thesis: a cleaning step that runs without an error has not shown that it kept every value.

**Nothing is typed in this lecture.** On a prediction slide you write your answer and one reason, in silence; we vote; then the next slide shows what it printed.

# Bronze, silver, gold

every layer is made by code from the layer below; every run makes the same tables



## Watch — look before you load

The **lecture file** is `data/raw/env_air_gge_ghg.tsv`, greenhouse-gas emissions. (The Lab 5 project has a second file: that one is the lab's.)

```
$ head -n 3 data/raw/env_air_gge_ghg.tsv | cut -c 1-72
freq,unit,airpol,src_crf,geo\TIME_PERIOD      2010      2011      2012      2013      2014      2
A,MIO_T,GHG,CRF1,AT      59.33986      57.11387      54.89359      55.09796      51.36292      53
A,MIO_T,GHG,CRF1,BE      99.26363      89.79709      88.92666      88.69095      82.7348      86.
$ grep 'CRF1B1,AT' data/raw/env_air_gge_ghg.tsv | cut -c 1-60
A,MIO_T,GHG,CRF1B1,AT      0 d      0 d      0 d      0 d      0 d      0 d      0 d      0 d      0 d      0
A,THS_T,GHG,CRF1B1,AT      0 d      0 d      0 d      0 d      0 d      0 d      0 d      0 d      0 d      0
```

- The first column packs five fields into one: `freq,unit,airpol,src_crf,geo\TIME_PERIOD`
- Each **year** is a **column**. Each cell is **text**: a number, a number and a flag, or `:`

## Predict, then watch — bronze: the file, as text

```
con.sql("""
    CREATE OR REPLACE VIEW bronze AS
    SELECT * FROM read_csv('data/raw/env_air_gge_ghg.tsv',
                           delim = '\t', all_varchar = true, names = ['series'])
""")
# and without all_varchar: what type does DuckDB guess for the year columns?
con.sql("DESCRIBE SELECT * FROM read_csv('data/raw/env_air_gge_ghg.tsv', delim = '\t')")
```

`all_varchar = true`: every cell arrives as text. `names = ['series']`: a short name for the packed first column.

Predict: **without** `all_varchar`, DuckDB guesses a type for each column. Would it read the year columns as numbers, or as text? Why?

## Bronze: the file, as text — what it printed

```
cell 2 con.sql("""
    CREATE OR REPLACE VIEW bronze AS
    SELECT * FROM read_csv('data/raw/env_air_gge_ghg.tsv',
                           delim = '\t', all_varchar = true, names = ['series'])
    """)
# The prediction: WITHOUT all_varchar, what type would DuckDB guess for the year columns?
con.sql("""
    SELECT column_name, column_type
    FROM (DESCRIBE SELECT * FROM read_csv('data/raw/env_air_gge_ghg.tsv', delim = '\t'))
    WHERE column_name IN ('2010', '2022', '2024')
    """).df()
```

|   | column_name | column_type |
|---|-------------|-------------|
| 0 | 2010        | VARCHAR     |
| 1 | 2022        | VARCHAR     |
| 2 | 2024        | VARCHAR     |

## Watch — long: one observation per series and year

```
cell 3 con.sql("""
CREATE OR REPLACE VIEW long AS
SELECT split_part(series, ',', 1) AS freq,
       split_part(series, ',', 2) AS unit,
       split_part(series, ',', 3) AS airpol,
       split_part(series, ',', 4) AS src_crf,
       split_part(series, ',', 5) AS geo,
       CAST(year AS INTEGER)      AS year,
       cell
FROM read_csv('data/raw/env_air_gge_ghg.tsv', delim = '\t', all_varchar = true, names = ['series'])
UNPIVOT INCLUDE NULLS (cell FOR year IN (COLUMNS('^[\d-]{4}$'))))
""")
con.sql("SELECT COUNT(*) AS observations FROM long").df()
```

|   | observations |
|---|--------------|
| 0 | 156660       |

Bronze: one row per series, the years across. Long: one row per observation, one series in one year.

**INCLUDE NULLS** : an empty cell stays a row.

## Watch — the census of the flag

```
cell 4 con.sql("""  
    SELECT split_part(cell, ' ', 2) AS flag, COUNT(*) AS n  
    FROM long  
    GROUP BY flag  
    ORDER BY n DESC  
""").df()
```

|   | flag | n      |
|---|------|--------|
| 0 |      | 117530 |
| 1 | m    | 38798  |
| 2 | d    | 304    |
| 3 | @C   | 28     |

The census first, always: before you convert a column, read every value in it.

## Predict, then watch — every cell that is not a number

```
con.sql("SELECT cell, COUNT(*) AS n FROM long WHERE cell LIKE ':%' GROUP BY cell ORDER BY n DESC").df()
```

`:` means "no number here".

Predict: is a `:` cell one kind of cell, or several? Why?

## Every cell that is not a number — what it printed

cell 5

```
con.sql("""  
    SELECT cell, COUNT(*) AS n  
    FROM long  
    WHERE cell LIKE ':%'  
    GROUP BY cell  
    ORDER BY n DESC  
""").df()
```

|   | cell | n     |
|---|------|-------|
| 0 | : m  | 38798 |
| 1 | : d  | 90    |
| 2 | :    | 86    |
| 3 | : @C | 28    |

## Predict, then watch — **CAST**: numbers, or an error?

```
con.sql("""
    SELECT geo, src_crf, year, CAST(cell AS DOUBLE) AS value
    FROM long
    WHERE unit = 'MIO_T'
""").df()
```

**CAST(x AS DOUBLE)** turns text into a number.

Predict: a column of numbers, or an error? Why?

## CAST — what it printed

cell 6a

```
try:
    con.sql("""
        SELECT geo, src_crf, year, CAST(cell AS DOUBLE) AS value
        FROM long
        WHERE unit = 'MIO_T'
        """).df()
except duckdb.Error as e:
    print("DuckDB refused:", str(e).splitlines()[0])
```

DuckDB refused: Conversion Error: Could not convert string ': m' to DOUBLE when casting from source column cell

## Predict, then watch — a first silver: one unit, one cast

```
con.sql("""
    CREATE OR REPLACE VIEW silver_first AS
    SELECT geo, src_crf, year, TRY_CAST(cell AS DOUBLE) AS value
    FROM long
    WHERE unit = 'MIO_T'
""")
con.sql("SELECT COUNT(*) AS rows, COUNT(value) AS with_a_value FROM silver_first").df()
```

`TRY_CAST` is `CAST` that never errors: text it cannot read becomes `NULL`, silently. This runs with no error, and it keeps one unit of two: half the rows of `long`.

Predict: **did every number Eurostat wrote survive the cast?** Yes or no, and why?

**Your Lab 5 skeleton contains a parser like this one, for the waste file.**

## A first silver: one unit, one cast — what it printed

cell 6b

```
con.sql("""
    CREATE OR REPLACE VIEW silver_first AS
    SELECT geo, src_crf, year, TRY_CAST(cell AS DOUBLE) AS value
    FROM long
    WHERE unit = 'MIO_T'
""")
con.sql("SELECT COUNT(*) AS rows, COUNT(value) AS with_a_value FROM silver_first").df()
```

|   | rows  | with_a_value |
|---|-------|--------------|
| 0 | 78330 | 58722        |

## Watch — what did the cast swallow?

```
cell 7 con.sql("""  
    SELECT cell, COUNT(*) AS n  
    FROM long  
    WHERE TRY_CAST(cell AS DOUBLE) IS NULL      -- the cast gave nothing  
          AND split_part(cell, ' ', 1) <> ':'    -- but Eurostat wrote a number  
    GROUP BY cell  
""").df()
```

|   | cell | n   |
|---|------|-----|
| 0 | 0 d  | 214 |

The answer to the last prediction: the cells where the cast gave nothing, and the source gave something.

## Predict, then watch — zeros, and the naive cast

**Solid-fuel fugitive emissions ( CRF1B1 ), 2022, the 31 countries.** Count the countries with zero emissions, and average them, once with the naive cast and once with the number part of the cell:

```
COUNT(*) FILTER (WHERE TRY_CAST(cell AS DOUBLE) = 0) AS zero_naive_cast,  
COUNT(*) FILTER (WHERE TRY_CAST(split_part(cell, ' ', 1) AS DOUBLE) = 0) AS zero_in_the_file
```

Predict: with the naive cast, does the count of zeros go up, down, or stay the same? And the sum? Why?

The full query: lecture notebook, cell 8.

## Zeros, and the naive cast — what it printed

cell 8

```
con.sql("""
    SELECT COUNT(*) FILTER (WHERE TRY_CAST(cell AS DOUBLE) = 0) AS zero_naive_cast,
           COUNT(*) FILTER (WHERE TRY_CAST(split_part(cell, ' ', 1) AS DOUBLE) = 0) AS zero_in_the_file,
           AVG(TRY_CAST(cell AS DOUBLE)) AS average_naive_cast,
           AVG(TRY_CAST(NULLIF(split_part(cell, ' ', 1), ':') AS DOUBLE)) AS average_in_the_file
    FROM long
    WHERE unit = 'MIO_T' AND year = 2022 AND src_crf = 'CRF1B1' AND geo <> 'EU27_2020'
""").df()
```

|   | zero_naive_cast | zero_in_the_file | average_naive_cast | average_in_the_file |
|---|-----------------|------------------|--------------------|---------------------|
| 0 | 11              | 12               | 1.032797           | 0.999481            |

## Watch — the number, the flag, and the cast

```
cell 9 con.sql("""
CREATE OR REPLACE VIEW parsed AS
SELECT *,
        NULLIF(split_part(trim(cell), ' ', 1), ':') AS number_text,
        NULLIF(split_part(trim(cell), ' ', 2), '') AS flag
FROM long
""")
con.sql("""
SELECT COUNT(*) AS swallowed FROM parsed
WHERE number_text IS NOT NULL AND TRY_CAST(number_text AS DOUBLE) IS NULL
""").df()
```

|   | swallowed |
|---|-----------|
| 0 | 0         |

Three moves: split, **NULLIF** the colon, cast only the number. Then the same count of what the cast swallowed.

## Predict, then watch — one country, one year, one sector

Germany's total ( T0TX4\_MEM0 ), 2022:

```
SELECT unit, TRY_CAST(cell AS DOUBLE) AS value
FROM long
WHERE geo = 'DE' AND year = 2022 AND src_crf = 'T0TX4_MEM0'
```

Predict: how many rows? And what would SUM(value) of them mean? Why?

The query is cell 10 of the lecture notebook.

## One country, one year, one sector — what it printed

```
cell 10 con.sql("""  
    SELECT unit, TRY_CAST(cell AS DOUBLE) AS value  
    FROM long  
    WHERE geo = 'DE' AND year = 2022 AND src_crf = 'TOTX4_MEMO'  
    """).df()
```

|   | unit  | value        |
|---|-------|--------------|
| 0 | MIO_T | 749.26642    |
| 1 | THS_T | 749266.42000 |

## Watch — the EU among its members

Total greenhouse gases ( T0TX4\_MEM0 ), 2022, million tonnes. Three sums: the EU27\_2020 row; the 27 member countries; every row.

```
cell 11 con.sql("""
    SELECT SUM(TRY_CAST(cell AS DOUBLE)) FILTER (WHERE geo = 'EU27_2020') AS eu27_row,
           SUM(TRY_CAST(cell AS DOUBLE)) FILTER (
               WHERE geo NOT IN ('EU27_2020', 'CH', 'IS', 'NO', 'TR')) AS sum_of_27_members,
           SUM(TRY_CAST(cell AS DOUBLE)) AS every_row
    FROM long
    WHERE unit = 'MIO_T' AND year = 2022 AND src_crf = 'T0TX4_MEM0'
""").df()
```

|   | eu27_row   | sum_of_27_members | every_row  |
|---|------------|-------------------|------------|
| 0 | 3359.59802 | 3359.598          | 7375.81718 |

Block 1's lesson in different clothes: a **group of countries** sits among the countries. In silver it is kept and marked: **is\_aggregate** .

# Predict, then watch — every stage has a named key

| stage                                           | one row is                       | key                                      |
|-------------------------------------------------|----------------------------------|------------------------------------------|
| raw (bronze)                                    | one series, years across         | (freq, unit, airpol, src_crf, geo)       |
| long                                            | one observation                  | (freq, unit, airpol, src_crf, geo, year) |
| silver — after freq = A, airpol = GHG, one unit | ?                                | ?                                        |
| gold                                            | one country-year, or one country | (geo, year) or geo                       |

```
SELECT COUNT(*), COUNT(DISTINCT (geo, year, src_crf)), COUNT(DISTINCT (geo, year)) FROM silver_first
```

Predict: is (geo, year) the key of silver\_first ? Yes or no, and why?

## Every stage has a named key — what it printed

cell 12

```
con.sql("""
    SELECT COUNT(*)                                AS rows,
           COUNT(DISTINCT (geo, year, src_crf)) AS geo_year_sector,
           COUNT(DISTINCT (geo, year))          AS geo_year
    FROM silver_first
""").df()
```

|   | rows  | geo_year_sector | geo_year |
|---|-------|-----------------|----------|
| 0 | 78330 | 78330           | 480      |

# The accounting: every observation lands in exactly one place

Counted at the **observation** grain, decided in this order — the first rule that matches wins:

|   | destination | rule                                                                            | this file |
|---|-------------|---------------------------------------------------------------------------------|-----------|
| 1 | rejected    | the cell cannot be read — written to <code>rejects.csv</code> with a reason     | 0         |
| 2 | excluded    | not the unit silver keeps ( <code>THS_T</code> )                                | 78,330    |
| 3 | aggregate   | <code>EU27_2020</code> — kept in silver, <code>is_aggregate</code> = true       | 2,490     |
| 4 | retained    | everything else — <code>:</code> is retained as <code>NULL</code> , and counted | 75,840    |

**156,660 in = 0 + 78,330 + 2,490 + 75,840**, printed at the end of every run. With the naive cast, 107 of the retained — the `0 d` zeros in `MIO_T` — would be `NULL` , and nobody would have said so.

## Predict, then watch — the precedence, as a **CASE**, counted with **FILTER**

```

CASE
  WHEN cell IS NULL OR trim(cell) = '' THEN 'rejected'
  WHEN number_text IS NOT NULL
        AND TRY_CAST(number_text AS DOUBLE) IS NULL THEN 'rejected'
  WHEN unit <> 'MIO_T' THEN 'excluded'
  WHEN geo = 'EU27_2020' THEN 'aggregate'
  ELSE 'retained'
END AS destination
...
COUNT(*) FILTER (WHERE destination = 'rejected') AS rejected,  -- one column per destination

```

Predict: the five counts come from this one **CASE**. **Could they ever fail to add up** to the observations in? Yes or no, and why?

The full query: lecture notebook, cells 13a (the **CASE**, as a view) and 13b (the counts).

## The precedence, as a **CASE**, counted with **FILTER** — what it printed

cell 13b

```
con.sql("""
    SELECT COUNT(*)                                AS observations_in,
           COUNT(*) FILTER (WHERE destination = 'rejected') AS rejected,
           COUNT(*) FILTER (WHERE destination = 'excluded') AS excluded,
           COUNT(*) FILTER (WHERE destination = 'aggregate') AS aggregate,
           COUNT(*) FILTER (WHERE destination = 'retained') AS retained,
           COUNT(*) FILTER (WHERE destination = 'retained' AND number_text IS NULL) AS retained_not_available
    FROM sorted
""").df()
```

|   | observations_in | rejected | excluded | aggregate | retained | retained_not_available |
|---|-----------------|----------|----------|-----------|----------|------------------------|
| 0 | 156660          | 0        | 78330    | 2490      | 75840    | 19414                  |

## The data dictionary is part of silver

| column       | type    | meaning                                             | missing means       | made from                          |
|--------------|---------|-----------------------------------------------------|---------------------|------------------------------------|
| geo          | VARCHAR | the country ( EL Greece) or EU27_2020               | never               | 5th field of series                |
| src_crf      | VARCHAR | the source sector ( CRF1B1 solid fuels, fugitive)   | never               | 4th field of series                |
| value        | DOUBLE  | million tonnes of CO <sub>2</sub> -equivalent       | Eurostat wrote<br>: | the number in the cell             |
| flag         | VARCHAR | Eurostat's flag: d definition differs, m missing... | no flag             | the letters after the number, kept |
| is_aggregate | BOOLEAN | true for EU27_2020                                  | never               | geo                                |

Plus: one row per (geo, year, src\_crf) ; one unit, MI0\_T . Lab 5 ships one: docs/dictionary.md .

## The pandas side note

The same moves, in the words Coding 1 will use:

| SQL (this course)                         | pandas                                                                                                            |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>split_part(cell, ' ', 1)</code>     | <code>s.str.split(" ", n=1, expand=True)</code>                                                                   |
| <code>TRIM, LOWER</code>                  | <code>s.str.strip().str.lower()</code>                                                                            |
| <code>TRY_CAST(cell AS DOUBLE)</code>     | <code>pd.to_numeric(s, errors="coerce")</code> — same swallow: <code>"628 e"</code> → <code>NaN</code> , no error |
| <code>UNPIVOT</code>                      | <code>df.melt(...)</code>                                                                                         |
| <code>strptime / CAST(... AS DATE)</code> | <code>pd.to_datetime(s, format="%Y-%m-%d")</code>                                                                 |

## Git thread — raw is committed, generated is ignored

```
# .gitignore (in every pipeline project)
data/silver/
data/gold/
output/
```

- `data/raw/` is committed: it is the evidence, and it never changes
- `data/silver/` , `data/gold/` are **not**: the pipeline makes them, so the pipeline can make them again
- A silver file in `git status` means you edited raw data, or you are about to commit what a script should make
- Commit the fix with the accounting line in the message

## AI thread — ask what it could not read

When an assistant cleans a file for you, the code it hands back runs. Ask it three things:

1. What did the cast do to cells that are not just a number?
2. Write the query that counts them.
3. Where do those cells go — silver, a rejects file, or nowhere?

Then run the count yourself. **The assistant cleans what it can see; the accounting line shows what it dropped.**

## The class of failure

A conversion that returns `NULL` instead of an error.

- `TRY_CAST`, `pd.to_numeric(errors="coerce")`, a date parse with `try`
- No error. Every row still there. **Values gone.**
- A row count cannot see it. A total may not see it — this file's swallowed values were all zeros.
- Only one thing sees it: **the count of cells where the cast gave nothing and the source gave something.**

## 5-minute stand-and-stretch — start the clone first

In the **terminal** (Git Bash on Windows, Terminal on macOS), in your course folder, then take your break:

```
git clone https://github.com/earino/ecbs5294-lab05-cleaning.git  
cd ecbs5294-lab05-cleaning  
uv sync
```

It downloads while you stretch. Water, window, hallway. Back in five minutes, then the lab. **The first 20 minutes are AI-off.**

## Lab 5 — The clean table that is not clean

Your Lab 5 project: `pipeline.py` runs `scripts/clean.py` (raw → silver) and `scripts/report.py` (silver → gold). The file is the **waste file**, `data/raw/env_wasmun.tsv`.

The report says **15 places** have a 2024 value for waste generated per person. Germany is blank. Eurostat's file has a number for Germany in 2024.

**First minute:** check the clone you started ran — VS Code → **File** → **Open Folder...** → `ecbs5294-lab05-cleaning`.

1. `uv run python pipeline.py` — read the report
2. `notebooks/diagnose.ipynb` — look first; **question 5 is yours**, from an empty cell; paste into `DIAGNOSIS.md` before you change anything
3. `scripts/clean.py` — section C, then the four destinations in E; the key check in F after the note

**By minute 22:** section C rewritten, and a report with more than 15 places. Not there? Hand up.

**AI-off for the first 20 minutes.** No stretch task. The neighbour slide comes up on time, finished or not.

## The last ten minutes — explain it to a neighbour

**Speaker, about a minute, pointing at the screen, not reading the note:** symptom → cause → evidence → change → verification.

**Listener asks:**

1. Show me the query that proves it.
2. Why was it wrong, not just where?
3. Next year's file uses a flag letter you have not seen. What does your parser do with it, and what tells you?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` to the Lab 5 checkpoint on Moodle.